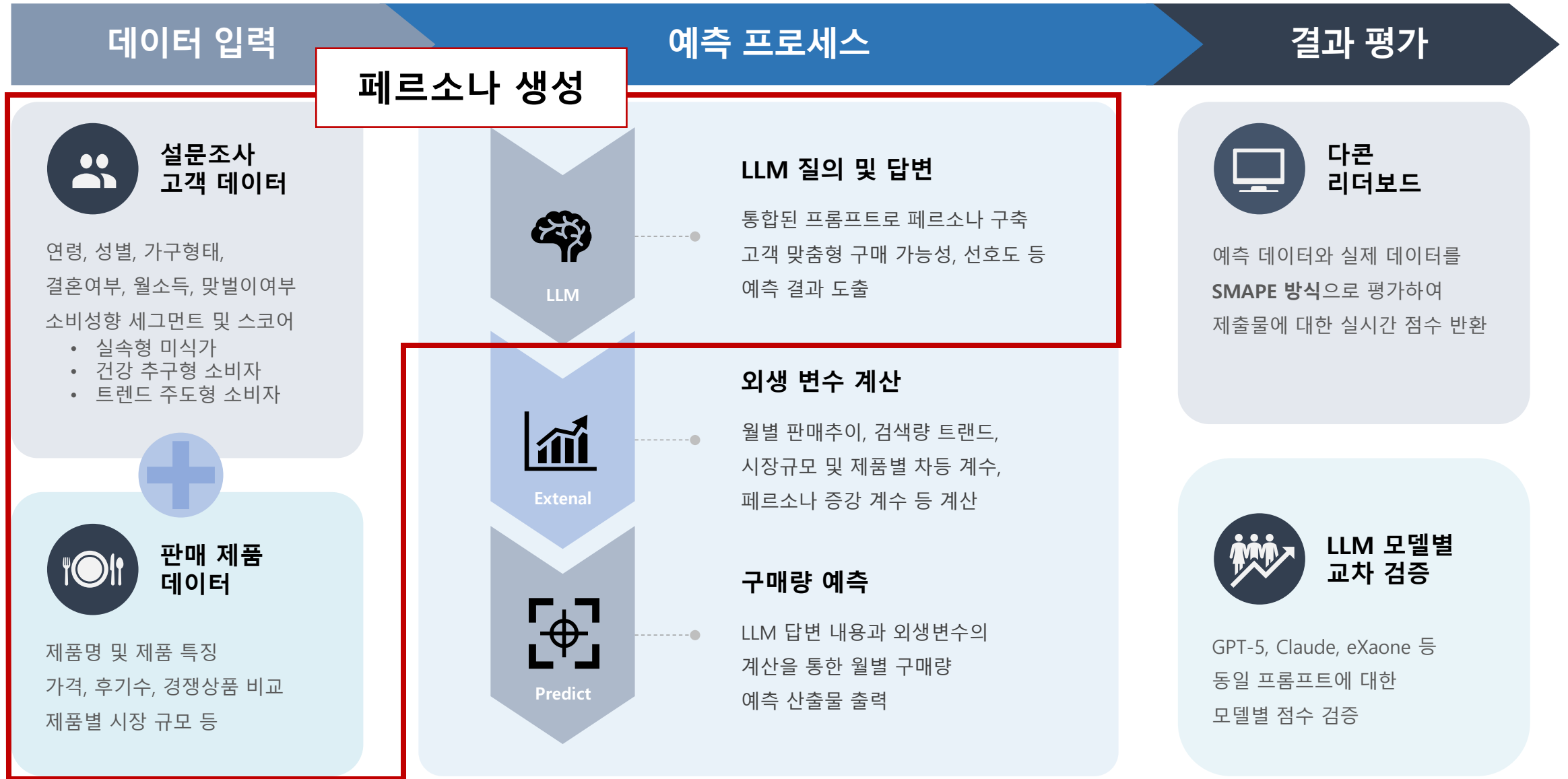


시뮬레이션 계획



페르소나 생성 프롬프트

system prompt = "다음 조건을 만족하는 페르소나 정보를 JSON 배열 형식 (UTF-8, 인코딩된 따옴표 없음)으로 정확하게 생성하세요. 반드시 JSON 배열만 출력하세요."

user prompt = "" [Cluster Catalog: DO NOT DEVIATE]

0: {'label': '실속형 미식가', 'description': '편리성을 중시하면서도 새로운 맛과 제품을 시도하는 데 적극적인 소비자 그룹입니다. 가격에 민감하기보다는 효율성과 맛을 동시에 추구합니다.'}

1: {'label': '건강 추구형 소비자', 'description': '가격이나 브랜드에 크게 구애받지 않고 건강을 최우선으로 고려하는 프리미엄 소비자 그룹입니다. 건강과 편리성을 모두 만족시키는 제품에 기꺼이 지갑을 엽니다.'}

3: {'label': '트렌드 주도형 소비자', 'description': '자신의 취향과 경험을 중시하는 소비자 그룹입니다. 단순히 배를 채우는 것 이상의 가치를 추구하며, 새로운 제품을 가장 먼저 경험하고 공유하려는 경향이 강합니다.'}

[Constraints]

- 생성되는 {batch_size}명의 페르소나는 cluster 0, 1, 3을 골고루 포함하도록 할 것
- 성별, 연령대, 직업군의 다양성이 유지되어야 하며, 특정 집단으로 편중되지 않도록 유의할 것
- meta.label / meta.description은 반드시 위 Catalog에서 복사할 것 (변형 금지)

{few_shot_block}

이제 위 조건을 만족하는 새로운 페르소나 {batch_size}개를 JSON 배열 형식으로 생성해주세요.

제품 ID: {product_name}

페르소나 번호: {product_name}_{start_index}

[출력 형식 및 속성 예시값] (JSON 배열 형식! 단일 객체 아님)

아래는 페르소나를 생성할 때 따르는 속성 목록과 형식 예시입니다. 단, 속성 값은 반드시 아래 예시값 중에서 다양하게 조합하여 사용하세요.

```
```json
{
 "persona_key": "{product_name}_{start_index}",
 "attributes": {
 "gender": "남자 또는 여자 중 선택",
 "age": "20대 / 30대 / 40대 / 50대 / 60대 이상 중 선택",
 "job": "관리자 / 군인 / 기능원 및 관련 기능 종사자 / 농림어업 숙련 종사자 / 단순노무 종사자 / 사무 종사자 / 서비스 종사자 / 전문가 및 관련 종사자 / 판매 종사자 / 장치·기계 조작 및 조립 종사자 / 주부 / 취업 준비 중 / 학생 중 선택",
 "education": "고졸(대학 재학 포함) / 대학교 졸업(전문대졸/대학원생 포함) / 대학원 졸업 이상 / 중졸 이하 중 선택",
 "region": "강원도 / 경기도 / 경상남도 / 경상북도 / 광주광역시 / 대구광역시 / 대전광역시 / 서울특별시 / 세종특별자치시 / 울산광역시 / 인천광역시 / 전라남도 / 전라북도 / 제주특별자치도 / 충청남도 / 충청북도 중 선택",
 "household": "1인 가구 / 1세대가족 / 2세대가족 중 선택",
 "marriage": "기혼 / 미혼(사별, 이혼 포함) 중 선택",
 "income_status": "맞벌이 함 / 맞벌이 하지 않음 중 선택",
 "income_month": "100만원 미만 ~ 1,000만원 이상 중 선택",

 "brand_loyalty_scaled": "0.0 ~ 1.0 사이의 실수값",
 "cooking_convenience_scaled": "0.0 ~ 1.0 사이의 실수값",
 "health_orientation_scaled": "0.0 ~ 1.0 사이의 실수값",
 "hmr_preference_scaled": "0.0 ~ 1.0 사이의 실수값",
 "premium_orientation_scaled": "0.0 ~ 1.0 사이의 실수값",
 "price_sensitivity_scaled": "0.0 ~ 1.0 사이의 실수값",
 "variety_seeking_scaled": "0.0 ~ 1.0 사이의 실수값"
 },

 "meta": {
 "cluster": "0, 1, 3 중 택일",
 "label": "해당 cluster에 맞는 label을 Catalog에서 복사",
 "description": "해당 cluster에 맞는 description을 Catalog에서 복사"
 },
}
```
```

페르소나 생성 프로세스

1. 초기 설정 및 라이브러리 불러오기

- 필요한 라이브러리 불러오기
- .env 파일에서 OpenAI API키와 모델 불러오기

2. 데이터 로드 및 전처리

- Few-shot 예시로 쓸 '페르소나 가중치 변환.jsonl' 파일에서 페르소나 데이터를 불러와 리스트 형태로 저장
- 'product_info.json' 파일에서 제품 정보를 불러와 딕셔너리로 저장
- 제품 특성과 페르소나 정보를 결합하여 모델 입력에 맞게 데이터 정리

3. 함수 정의

- 클러스터 카탈로그를 정의하고, 클러스터 메타데이터를 강제 설정하는 enforce_cluster_meta 함수를 정의
- Few-shot 학습 예시를 다양하게 샘플링하는 sample_diverse_examples 함수와 이를 블록 형태로 만드는 build_fewshot_block 함수를 정의
- 제품과 페르소나 정보를 결합하여 LLM에 보낼 최종 프롬프트를 만드는 build_conditioned_prompt 함수를 정의

4. Few-shot 예시 확인

- sample_diverse_examples 함수를 호출하여 10개의 예시 페르소나를 무작위로 선택하고 출력

5. AI 모델 예측 및 데이터 처리

- 결과 파일을 저장할 디렉토리를 생성
- build_fewshot_block 함수를 호출하여 few-shot 프롬프트 블록 생성
- 5명씩의 페르소나 배치를 생성하기 위해 100회 반복하는 루프를 실행
- client.chat.completions.create를 호출하여 LLM에 최종 프롬프트 전달
- 각 제품별 페르소나를 JSON 부분을 추출하고 json.loads로 파싱
- enforce_cluster_meta 함수를 호출하여 정보를 보정
- 처리된 페르소나를 JSONL 형식으로 저장

enforce_cluster_meta(persona: Dict[str, Any]) -> Dict[str, Any])

생성된 페르소나의 meta.cluster값을 기준으로 meta.label과 meta.description을 자동으로 채워주는 역할

LLM이 생성한 페르소나 결과에서 클러스터 ID가 존재할 경우, 이를 사전에 정의된 클러스터 카탈로그에서 찾아서 레이블과 설명을 추가

cluster_catalog_block() -> str

클러스터 ID → 라벨/설명 매핑표를 텍스트 블록 형식으로 출력하는 역할
프롬프트에 포함시켜 LLM이 클러스터 레이블 의미를 이해하도록 도와줌

meta_constraints_block() -> str

페르소나 생성 시 meta 정보에 대한 제약 조건 설명을 출력

operation_tips_block() -> str

LLM에 줄 추가적인 생성 지침 또는 유의사항을 설명

```
def enforce_cluster_meta(persona: Dict[str, Any]) -> Dict[str, Any]:
    """
    LLM이 생성한 persona에서 meta.cluster에 맞춰
    meta.label과 meta.description을 카탈로그 값으로 강제 세팅.
    """

    meta = persona.get("meta", {})
    cluster = meta.get("cluster")

    if isinstance(cluster, bool):
        cluster = int(cluster)
    if isinstance(cluster, (int, float)):
        cluster = int(cluster)
    else:
        raise ValueError("meta.cluster must be an integer.")

    if cluster not in CLUSTER_CATALOG:
        raise ValueError(f"Unknown cluster: {cluster}")

    meta["label"] = CLUSTER_CATALOG[cluster]["label"]
    meta["description"] = CLUSTER_CATALOG[cluster]["description"]
    return persona
```

```
def cluster_catalog_block() -> str:
    lines = ["[Cluster Catalog: DO NOT DEVIATE]"]
    for k, v in CLUSTER_CATALOG.items():
        lines.append(f"{k}:")
        lines.append(f"  label: #{v['label']}###")
        lines.append(f"  description: #{v['description']}###")
    return "\n".join(lines)

def meta_constraints_block() -> str:
    return (
        "[Meta Constraints]\n"
        "- #{meta.cluster}가 존재하면, #{meta.label}과 #{meta.description}은 반드시 위 Catalog의 동일한 값을 그대로 복사한다.\n"
        "- 재해석 · 변형 · 요약 금지. 오직 복사."
    )

def operation_tips_block() -> str:
    return (
        "[Operation Tips]\n"
        "1) few-shot 안에도 3개의 서로 다른 cluster를 포함해, cluster가 바뀌면 label/description도 달라짐을 암시적으로 학습시킨다.\n"
        "2) 서버 보강(enforce_cluster_meta)을 항상 거친다. 프롬프트만 믿지 않는다.\n"
        "3) 클러스터 설명을 업데이트할 때는 Catalog 한 곳만 수정해 전체 파이프라인의 일관성을 유지한다.\n"
        "4) 예시 및 생성 결과에는 연령대(20대, 30대, 40대, 50대, 60대 이상), 성별, 직업의 다양성이 반영되도록 주의한다."
    )
```

sample_diverse_examples(personas:
List[Dict[str, Any]], k=10)-> List[Dict[str, Any]]

LLM에게 줄 다양한 페르소나 중에서

cluster, age, gender를 기준으로 중복되지 않도록

서로 다른 특징을 가진 예시 k개를 샘플링하는 함수

```
def sample_diverse_examples(personas: List[Dict[str, Any]], k: int = 10) -> List[Dict[str, Any]]:
    selected, seen_keys = [], set()
    clusters = [0, 1, 3]
    for o in clusters:
        oands = [p for p in personas if p["meta"]["cluster"] == o]
        random.shuffle(oands)
        for p in oands:
            key = (o, p["attributes"]["age"]["value"], p["attributes"]["gender"]["value"], p["attributes"]["job"]["value"])
            if key not in seen_keys:
                selected.append(p)
                seen_keys.add(key)
            break
    remainder = [p for p in personas if p not in selected]
    random.shuffle(remainder)
    for p in remainder:
        key = (p["meta"]["cluster"], p["attributes"]["age"]["value"], p["attributes"]["gender"]["value"], p["attributes"]["job"]["value"])
        if key not in seen_keys:
            selected.append(p)
            seen_keys.add(key)
        if len(selected) >= k:
            break
    return selected
```

sbuild_fewshot_block(personas: List[Dict[str,
Any]], k=10) -> str

sample_diverse_examples 함수를 호출해서 다양성을 보장
한 k개의 페르소나를 골라 각 페르소나에 대해
enforce_cluster_meta 함수를 적용

persona_key, attributes, meta항목만 추려서 dict 구성하고
json.dumps로 포맷하여 전체 block에 Markdown 코드블럭
(``json)형태로 추가

```
def build_fewshot_block(personas: List[Dict[str, Any]], k=10) -> str:
    examples = sample_diverse_examples(personas, k)
    block = "다음은 소비자 페르소나 예시입니다:\n\n"
    for p in examples:
        p = enforce_cluster_meta(p)
        flat = {
            "persona_key": p["persona_key"],
            "attributes": {k: v["value"] for k, v in p["attributes"].items()},
            "meta": p["meta"]
        }
        block += "```\njson\n" + json.dumps(flat, ensure_ascii=False, indent=2) + "\n```\n\n"
    return block
```

build_conditioned_prompt(product_name: str,
few_shot_block: str, start_index: int,
batch_size: int = 5) -> str

product_name과 사전에 정의된 few-shot 예시
(few_shot_block)을 기반으로 persona_key가 start_index부
터 batch_size만큼 반복되는 LLM 프롬프트 텍스트를 주어진
JSON 형식에 맞게 생성하는 함수

```
def build_conditioned_prompt(product_name: str, few_shot_block: str, start_index: int, batch_size: int = 5) -> str:  
    return f"""  
[Cluster Catalog: DO NOT DEVIATE]  
0: {CLUSTER_CATALOG[0]}  
1: {CLUSTER_CATALOG[1]}  
3: {CLUSTER_CATALOG[3]}  
[Constraints]  
- 생성되는 {batch_size}명의 페르소나는 cluster 0, 1, 3을 골고루 포함하도록 할 것  
- 성별, 연령대, 직업군의 다양성이 유지되어야 하며, 특정 집단으로 편중되지 않도록 유의할 것  
- meta.label / meta.description은 반드시 위 Catalog에서 복사할 것 (변형 금지)  
  
{few_shot_block}  
이제 위 조건을 만족하는 새로운 페르소나 {batch_size}개를 JSON 배열 형식으로 생성해주세요.  
  
제품 ID: {product_name}  
페르소나 번호: {product_name}_{start_index}
```

[출력 형식 및 속성 예시값] (JSON 배열 형식! 단일 객체 아님)

아래는 페르소나를 생성할 때 따르는 속성 목록과 형식 예시입니다. 단, 속성 값은 반드시 아래 예시값 중에서 다양하게 조합하여 사용하세요.

```
'''json  
{  
  "persona_key": "{product_name}_{start_index}",  
  "attributes": {  
    "gender": "남자 또는 여자 중 선택",  
    "age": "20대 / 30대 / 40대 / 50대 / 60대 이상 중 선택",  
    "job": "관리자 / 군인 / 기능원 및 관련 기능 종사자 / 농업·어업·숙련 종사자 / 단순노무 종사자 / 사무 종사자 / 서비스 종사자 / 전문직 종사자",  
    "education": "고졸(대학 재학 포함) / 대학교 졸업(전문대졸/대학원생 포함) / 대학원 졸업 이상 / 중졸 이하 중 선택",  
    "region": "강원도 / 경기도 / 경상남도 / 경상북도 / 광주광역시 / 대구광역시 / 대전광역시 / 서울특별시 / 세종특별자치시 / 울산광역시",  
    "household": "1인 가구 / 1세대가족 / 2세대가족 중 선택",  
    "marriage": "기혼 / 미혼(사별, 미혼 포함) 중 선택",  
    "income_status": "맞벌이 함 / 맞벌이 하지 않음 중 선택",  
    "income_month": "100만원 미만 ~ 1,000만원 이상 중 선택",  
  
    "brand_loyalty_scaled": "0.0 ~ 1.0 사이의 실수값",  
    "cooking_convenience_scaled": "0.0 ~ 1.0 사이의 실수값",  
    "health_orientation_scaled": "0.0 ~ 1.0 사이의 실수값",  
    "hmr_preference_scaled": "0.0 ~ 1.0 사이의 실수값",  
    "premium_orientation_scaled": "0.0 ~ 1.0 사이의 실수값",  
    "price_sensitivity_scaled": "0.0 ~ 1.0 사이의 실수값",  
    "variety_seeking_scaled": "0.0 ~ 1.0 사이의 실수값"  
  },  
  "meta": {  
    "cluster": "0, 1, 3 중 택일",  
    "label": "해당 cluster에 맞는 label을 Catalog에서 복사",  
    "description": "해당 cluster에 맞는 description을 Catalog에서 복사"  
  },  
}  
'''
```

Few-shot 학습에 사용할 예시 페르소나를 선택하고 요약하는 코드

다시 실행해도 동일한 무작위 결과를 얻도록 시드 설정

sample_diverse_examples 함수를 호출하여 다양한 연령대, 성별, 직업을 가진 10개의 페르소나를 선택

1. product_info_list에 있는 각 제품을 순회하며 예측
2. build_fewshot_block 함수를 호출하여 AI 모델이 참조할 10개의 예시 페르소나 블록을 생성(시드를 설정하여 매번 같은 예시가 선택)
3. for i in range(0, 100, 5) 루프를 통해 총 100개의 페르소나를 5개씩 배치로 나누어 LLM에 요청
4. build_conditioned_prompt 함수로 product_info와 few-shot 예시를 결합한 최종 프롬프트 생성
5. client.chat.completions.create 함수를 사용하여 OpenAI API에 요청 (temperature=0.0으로 설정하여 답변의 일관성과 정확성을 높임)
6. 응답에서 불필요한 마크다운 코드블록을 제거하고 JSON 텍스트만 추출
7. json.loads를 사용하여 텍스트를 파이썬 객체로 변환
8. enforce_cluster_meta 함수를 호출하여 LLM이 생성한 페르소나의 클러스터 정보가 미리 정의된 CLUSTER_CATALOG의 값과 일치하도록 보정
9. 보정된 페르소나 데이터를 personas_output 리스트에 추가
10. 각 제품별 100개의 페르소나 데이터를 JSONL 형식으로 저장

```
random.seed(42)
fewshot_examples = sample_diverse_examples(personas, k=10)
summary = []
for p in fewshot_examples:
    summary.append({
        "persona_key": p.get("persona_key", "-"),
        "cluster": p["meta"]["cluster"],
        "gender": p["attributes"]["gender"]["value"],
        "age": p["attributes"]["age"]["value"],
        "job": p["attributes"]["job"]["value"],
        "region": p["attributes"].get("region", {}).get("value", "-"),
        "income_month": p["attributes"].get("income_month", {}).get("value", "-"),
    })

print(pd.DataFrame(summary).to_string(index=False))
```

```
os.makedirs("data/outputs/llm_outputs", exist_ok=True)
os.makedirs("data/outputs", exist_ok=True)

monthly_rows = []

for idx, product in enumerate(product_info_list):
    product_name = product["product_name"]
    release_date = product["release_date"]

    random.seed(42)
    few_shot_block = build_fewshot_block(personas, k=10)
    personas_output = []

    for i in range(0, 100, 5): # 5행씩 batch 생성
        prompt = build_conditioned_prompt(product_name, few_shot_block, start_index=i, batch_size=5)

        try:
            response = client.chat.completions.create(
                model=OPENAI_MODEL,
                messages=[
                    {"role": "system", "content": "다음 조건을 만족하는 페르소나 정보를 JSON 배열 형식 (UTF-8, 인코딩된 따옴표 없음)"},
                    {"role": "user", "content": prompt}
                ],
                temperature=0.0
            )

            content = response.choices[0].message.content.strip()
            if "```json" in content:
                content = content.split("```json")[1].split("```")[0].strip()
            elif "```" in content:
                content = content.split("```")[1].strip()

            parsed = json.loads(content)
            for j, persona in enumerate(parsed):
                persona["persona_key"] = f"{product_name}_{i + j}"
                persona = enforce_cluster_meta(persona)
                personas_output.append(persona)

        except Exception as e:
            print(f"[{product_name}] 오류 발생 (batch {i}~{i+4}): {e}")
            continue

    with open(f"data/outputs/llm_outputs/{product_name}_20_personas.jsonl", "w", encoding="utf-8") as f:
        for p in personas_output:
            f.write(json.dumps(p, ensure_ascii=False) + "\n")

    monthly_rows.append(generate_monthly_prediction(product_name, release_date))
```